UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA

FACOLTÀ DI INGEGNERIA

CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

A lambda architecture based solution for analytics

A.A 2014/2015

RELATORE

Valeria Cardellini

CANDIDATO

Luzzi Matteo Remo

CORRELATORE

Glenn Skare Pedersenn

Inserire la dedica

Contents

[1] At the time of installation I came across a bug due to a repository out of service. Refer to [5] for its solution

Acknowledgement

Thanks to myself

Abstract

Chapter 1

Introduction

1.1 Background

In this section we will give an overview of the foundation this thesis project is based upon. We will briefly introduct the big data concept and its analytics derivation. We will focus then on what it is the state of the art concerning software architectures for big data collection and analysis giving pros and cons for every analyzed solution.

1.1.1 Big Data

Big data in one of the trending terms in the IT field which increased of popularity from 2012 [1]. A lot of definition has been given to this term [2], focusing on different aspects regarding the data itself and the infrastructure needed to process it.
Despite the recent dramatic increase of popularity, big data is a concept already coined in ealry 2000s, L. Douglas first introduced the 3Vs concept regarding the new challenges that systems had to deal with: Volume, Velocity, Variety. The report focalized on what are the problematics due to the increasing size of data, its frequency at which it is produced and a wider range of representation formats. Since 2001, when this report has been published, the question about big data hasn't changed a lot. Of course now we have better approaches and tecnhologies, but also the big data concept changed.

Iot

1.2 Introduction to analytics

click stream importanza analytics

1.2.1 Why analytics

1.2.2 Analytics metrics

Defining appropriate indicators for web analytics is challenging. Example: user session definition among different analytics provider. Hopefully there is a standard defined in 2008 by Web analytics association. List of the most importat KPI standardized by WM associacion.

1.2.3 A concrete example: Google Analytics

1.3 State of the art for data system

As stated in the previous section, dealing with analytics system could be really misleaving, and normally KPI changes from company to another. Then a lot of companies prefer to build their own in-house analytics system rather than use a standard one (like GA). Analytics system are a particular case of Big Data system, then when deveoping such a system, we can apply the same theoretical background. Distributed systems should be preferred in place of a standalone architectures as they offer better performances due to scalability, but they has also some drawbacks: they are complex and less consistent due to the distributed nature. Currently big data systems fall into these two categories:

- *batch processing* of big volume of data with high latency
- *real-time processing* with short latency (milliseconds) keeping the data in-memory.

While batch solution has been famus in the last years, recentry we saw a trend in the real time processing, due also to the lower price for RAM Cite the article (Using In-Memory Analytics to Quickly Crunch Big Data)

1.3.1 CAP theorem limitation

CAP (Consistency-Availability-Partition) theorem states that, in every system, only two of the three previous features can be achieved. In presence of distributed systyems we are forced to have partitions, then the CAP theorem says that we could only choose consistency over availability or viceversa.This means that we can either have a system capable of always serving a user request sacrifying the consistency of the data, or a system that sometime isn't available, but gives out consistet data (between all the partitions).

1.3.2 Lambda Architecture

Data system are particularlly influenced by CAP theorem, so a different approach is needed in order to find a tradeoff between availability and consistency. As stated before, a system can perform batch or (nearly) real-time operations on its data. We will explain in the next sections how lambda architecture aims to find a balance between them.

Motivations and architecture

Lambda architecture is a architectural pattern for big data processing system proposed by Nathan Marz [4]. Its aim is to address to the problem of quering a data system giving result *as much as possible synchronized* with the input data. This imply an user to be able to query the system in real-time in order to obtain information either on what's happening now, or what happened one day ago or also one month ago.

According to its creator, this type of architecture has to fullfil the following requirements:

- robusness and fault tolerance agains hardware failuers and human mistakes
- low latency of read and updates operations
- scalability
- generalization in term of different use cases covered by such a system
- extensibility i.e. the possibiliy of adding new features easily
- ad-hoc queries support

Everything is based on the simple equation that $query = function(all\,data)$ [4], which means that a data system simply answers questions about a dataset, no matter how complex is the query. The problem here is that, when dealing with big data (gigabyte/petabytes), normally a query would take a considerable amount of time to be executed: in this case the result could not be consistent with the entire dataset, since while computing the result, new data might have been injected into the system. Unfortunately there is no such a single system capable to anwser to every kind of query *in real-time*.

The first concept introduced by lambda architecture, is to precompute query functions [6] in order be able to answer quickly to them. If an user now, queried a particular timeframe of data, the system would answer immediately by showing the precomputed result. Computing aggregation results imposes high latency, a typical instance of aggregation deals with data collected in the last hours or even days and it may take hours to complete, then the last step is how to fill the gap between the aggreted results and

the new data not covered by aggregations yet.

In order to solve this problem, lambda architecture proposes a three layers architecture composed by:

- Batch layer
- Real-time layer
- Serving layer

Every single data ingested into the system is replicated into and elaborated by either the batch and real-time layer in order to have intermediate results (We will refer to them as *batch* and *real-time views*. The outcomes are then merged into the serving layer and ready to be queried by end users. Each layer has different characteristics according to the role they fullfil in the overall architecture.

The batch layer is composed by two parts: a master dataset and a batch computation part. The master dataset holds the data coming into the system needed for calculating the aggregated results. Data are considered immutable, no updates or deletion are *in theory* allowed: data can be appeded into or read from the master dataset. We will show later how this is a great advantage in terms of consistency. The batch computation part reads from the master dataset and computes batch views. Accordin to the equation proposed my Marz [6], the bath layer would solve the equation $batch\ view = function(all\ data)$. For the latter operation two appoaches are possible: incremental and recomputation. The first approach implies an update of previous views in order to compute the current one, the second insted involves a total recomputation based only on the raw data stored in the master dataset. Recomputation algorimths are easier as they result in a simpler logic, while sometimes using incremental algorithms may speed up the whole process as it exploits the existence of already computed

results. The choice of the best approach really depends to the use cases the system has to address to. For instance, let's assume we want to calculate the total number of visit for a webpage over the time. It is easy to understand that for this use case an incremental approach is preferred over a total recomputation on the entire dataset everytime new data come. The difference in time tends to diverge according to the quantity of data the batch is computed on.

The real-time layer aims to fill the gap introduced by the high latency of the batch layer and it is responsible for providing results on the most recent data. It takes the same input of the batch layer, but instead of storing it in a dataset, elaborates it on-the-fly. It behaves as the batch layer, in terms of precomputing the same type of views, the difference is that it works only with a small portion of data, exacly the one which it is not covered by batch views. An incremental approach is generally preferred and real-time views are update as new data comes into the system. Formally it solves the equation $realtime\ view = function(realtime\ view, new\ data)$ [6]. Overall the design of the real-time layer is more complex of the batch layer, either for its incremental approach, either for the requirements it has to respect such as short latency. Fortunately it is only responsible for a short dataset and its views are eventually corrected by the batch ones, so normally a certain degree of approximation is here tollerate. We will go more in detail into this in the next section.

The serving layer is responsible for providing results to ad-hoc queries with low latency as a combination of batch and real-time views. It indexes results from the batch layer in order to serve them with low latency and it compesate the lack of batch views for the new data by quering the real-time ones. Hence it gives a final anwser to the problem of quering a data system by solving the equation $query = function(batch\ view, real\text{-}time\ view)$ [6]. It is the layer with the most complex logic as it has to merge different

views according to type of query submitted to the system. A simple case is when the data are time ordered and a particular query refer to a fixed timeframe: the serving layer has just to aggregate batch views with, if needed, real-time ones. Though this is just a possible use case and a general purpose implementation of the serving layer might be not trivial.

CAP theorem & LA

Even though the audacious title of Marz article about lambda architecture (*i.d. "How to beat CAP theorem"* [4], CAP theorem does still holds [7]. What lambda architecture aims to achive is the so-called eventual consistency. This means, that, after a certain timeframe, data are eventually consistent across all the partitions of a distributed systems.

We stated before that every single layer of the lamdba architecture has to respect different constraints due to their different role in the overall system. The master dataset have only to support creation and read, making then its data and immutable sources. This is a great advantage, in the sense that the system does not have to deal with updates, which are one of the cause of lack of consistency in distributed systems. The overall batch layer is eventually consistent in the sense new data are consistent with the query results only after a batch views has been calculated and the database for storing batch-views has to support random reads and batch writes.

The real-time layer instead, has to be higly available in order to compensate the latency introduce by the batch layer. In order to compute real-time views fast, ofter are used approximation algorithms rather than exact ones (for instance the evalutation of distinct records on set is oftet extimated using *hyperloglog* [8]) and the database used

to store the real-time views has to support either random writes and reads due to the incremental apporach used by this computation.
The overall lambda architecture based system, as we said, is eventually consistent. Approximations and mistakes made by the real-time layer are eventually corrected by the batch views.

Pros and Cons

One of the main probles lambda architecture wanted to solve is the ability of quering data in real-time. In a certain way this has been achive. High latency introduced by batch computation is compensated by real-time views and at everytime the system is able to respond to a query, with a certain degree of accuracy as previusly stated. This is indeed a great result, specially comparted to what we can achive with regular data system (real-time computation **or** batch computation).
Having an immutable source of data gives to the system robustness against failover : if an error happens on the batch layer, either an error in the processing logic or an hardware failure, the entire computation can be run over again without any data loss. Calculation of the final result as a serie of transformations of an immutable dataset is a concept also used in batch/micro batch computation framework like Spark [10] to achive *exacly once* message semantic.
The main weakness of such a system is without doubts its complexity. The serving layer is the one with the most complex logic as we explained before, specially in case of query not dependent only by a time frame. Also, under a developing point of view, at the moment it does not exisist a unified framework in order to share the same logic between different layers. An attempt of unification has been made with twitter summingbird project [9]: the aim is to provide an high level abstraction for defining the logic of a

job and then share it among different framework for batch and real-time computation. Nevertheless this is still a raw project rather than a production-ready solution. Another drawbacks is the lack of flexibility regarding the queries such a system can serve. One of the requisite of lambda architecture is the ability to handle *ad-hoc* queries, in the sense they depend on parametes known only at query-time. What lambda architecture lacks is the ability to serve arbitrary queries since it deals only with precomputed views. This is, anyway, a reasonable weakness as it is the tradeoff.[continue]

1.3.3 Alternatives to lambda architecture

Kappa Architecture

Real Time Streaming architecture

iot architecture

1.3.4 Architectures comparison

Chapter 2

Big Data tools

In this chapeter we will describe the tools used for implementing a working prototype based on the lambda architecture paradigm previously explained. The scope of this thesis is not to make a survey of modern tecnologies used for big data and here we will give just an overview highliting just the purpose and the main feature of each used tool. In the appendix the basic configurations for each tool are showed. For a better understaning of these tools, the reader is invited to read more resources about them.

2.1 Kafka

Apache Kafka [12] is a general purpose distributed publish-subscribe message system originally developed at Linkedin and then opensourced in 2011 and inserted in the Apache ecosystem. The architecture is composed by one or more kafka servers, also known as *brokers*, which rely on Apache Zookeeper for coordinations.

The key abstraction in Kafka is a topic: *producers* publish message to a topic and *consumers* subscribe to one or more topics. A single topic is composed by a set of partitions, distributed and replicated among all the brokers in the cluster. For each partition there is a primary replica and a set of slaves. Since normally there are more partitions than brokers, a broker can play the role of master some partitions and the role

of slave for others. On each partition are stored metadata about the position consumer subscribed to that topic, also called offsets. An offset represents a checkpoint for that consumer, messages till that point are assumed to be correctly processed and the next time the consumer makes a fetch request, the broker will forward messages starting from that offset. The consumer, anyway, is still able to reprocess already committed messages for instance in response to a failover event.

What it is interesting in Kafka is how messages are forwarded from the producer to the consumer thought the brokers. Unlike other message systems which process the messages in-memory, Kafka does a strong usage of the filesystem: messages are immediately stored on filesystem and not deleted when read by a consumer but retained according to a configurable period. This avoid any kind of data loss for every commited message, even in case of brokers failure. The only information retained by the broker is the position of the cosumer for that topic (which are the offsets for the partitions the topic is composed by).

Kafka allows to implement either a *queing* or a *publish-subscribe* logic introducing the abstraction of **consumer group**. Each consumer labels itself with a consumer group name and is it responsible for one or more partition of a topic. Kakfa ensures than no more than one consumer per group reads from a partition: if in the group there are more consumers than partition, some of them will stay idle. On the opposite, a consumer will read from more than one partition. If all the consumers belong to the same group, then the system works as a queue, balancing the load across the consumer group. If all the consumers have different group, then the system works as a standard publish-subscribe.

Having the concept of the consumer group implies a total order over messages within each partition: the order on how are messages are read inside a partition is the same

for every consumer who subscribed that partition.

As a conclusion, is worthy mention that Kafka implements a *at-least-once* message semantic in the following way:

- Producer: when publishing a message and having a network error, the producer can't be sure if the message has been correctly committed to the broker
- Consumer: a message is said to be fully processed (from a broker perspective) only when its offset has been committed. The best the consumer can do is, read the message, process it and then commit the offset. If the consumer crashes before committing the offset but after processing the message, the one taking over will read the same message.

With this observation in mind, is it a duty of the application to be idempotent.

2.2 Hadoop

2.2.1 HDFS

2.2.2 Oozie

Apache oozie[13] is a workflow schduler for Hadoop. Each workflow is composed by a sequence of action executed in sequential mode, which means that an action can't be executed until the previous one ended. The architecture of oozie is quite simple: (give architecture explaination). For installing and configuring oozie the reader can go to 6 Oozie workflow are build on a DAG (direct acyclic graph) stucture and are composed by a sequenct two possible type of *nodes*:

- control flow: nodes that control the start, the end and the execution of the workflow without executing any concrete action. Example of this nodes are: start control node, end control node, kill control node, fork control node.

- action: nodes that execute a task. The actions supported by the latest version (4.2.0 July 2015) are Map-Reduce, Pig, Java, Fs (filesystem), ssh, Spark

2.3 Spark

Apache Spark is a distributed framework for data computing. It is based on the same paradigm of Hadoop/Map-Reduce, where the computation is split across several nodes in the cluster (*map*) and the final result is given by aggregating the intermediate ones (*reduce*). Spark has been designed for the class of applications which need to iterate several time on the same dataset, such as machine learning and interactive analytics, such that it is possible to store intermetiate data in memory instead of recompute it at every step. For these type of application, it has been showed how spark outperforms a classic map-reduce paradigm by 10 times [14].

To be run in a distibuted mode, Spark needs a cluster manager for resources allocation across all the nodes. The architecture is composed by a primary node called *driver program* and a set of worker nodes running *executor* threads. When an application is submitted to the cluster, the driver program spreads it across the worker nodes and schedules task to be run by each executor in a isolate context (each executor runs its own JVM). In this way there is complete isoltation between several applications running on the same cluster, the drawback is that executors beloning to different applications can't share data between themselves and also within the same application they need the support of the driver program for sharing data using special objects such as *broadcast* and *accumulator* variables.

The main concept around spark is an abstraction over a phisical dataset called *RDD* [15] (resilient distributed dataset). RDDs are read-only, fault-tolerant parallel datas-

[1]https://spark.apache.org/docs/latest/cluster-overview.htmld

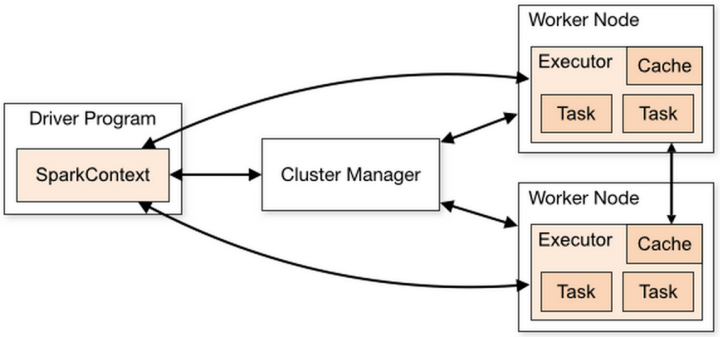

Figure 2.1: Spark cluster architecture[1]

tructures divided into one or more partition across a set of machines. They can be created from phisycal files, parallelizing a collection of elements or trasforming an existing RDD by applying a transformation (ex. map, filter...) or an action (ex reduce, count...). RDD are lazy by nature, in the sense that the driver program keeps in memory the list of transformations needed to compute it rather than execute them at each step. Only when an action requires the RDD to be returned to the driver program, the list of transformations is applied. By default, each transformed RDD is recalculated each time an action runs on it. It is possilble however, to cache in memory for faster successive accesses to it. This abstraction increases the robustness of the data model: in case of a partition failure, it can be reconstructed by applying the list of transformations and actions from the last RDD available in memory.

2.4 Storm

Apache Storm is a distributed real-time data processing framework created by Nathan Marz (the proposer of lambda architecture). Storm considers the data as a stream of *Tuple*, generated in special component called *spouts* and processed by one or more *bolt*. Spouts and bolt can be combined in a direct graph structure called *topology*.

A Storm cluster is composed by a master node and one ore more worker nodes. The master node runs a process called *Nimbus*: it is where the application submitted to the cluster and it is responsible for distributing the code, assign taks and check for failures. The worker nodes run one or more processes called *supervisor* in charge of communication with the master and of executing worker processes . Each worker process runs different parts of the topology in *tasks*. The master node communicates with the workers thought a *Zookepeer* cluster. Each component (spout or bolt) can be

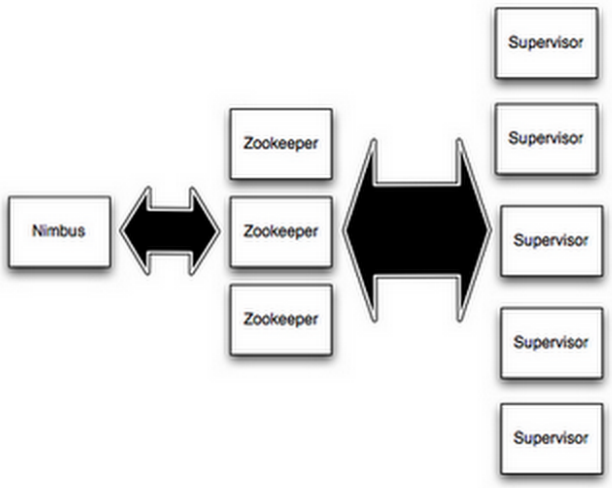

Figure 2.2: Storm cluster architecture[2]

replicated for load distibution. Whitin a worker process it is possible to run one or more *executor*, each *executor* finally runs one or more taks, with are just parts of the topology belonging to the same component. As stated before, the main abstraction in storm the stream of Tuple. Data flows from a component to the next one according to a partition policy: data can be distributed in shuffle mode, by a key, toward a unique component or replicated. Each mode fullfils a different use case: for instance a key-partition (*fileds grouping*) is usefull when we want to count the occurrency of a particular element in

[2]https://storm.apache.org/documentation/Tutorial.html

the data stream while a shuffle partition(*shuffle grouping*) is a better approach when the same operation has to be performed on each tuple, independently of the actual content of the tuple itself.

Storm guarantess by default a *at-least-once* message processing semantic. A topology can be launched with one ackers component, responsible of ensuring that a tuple is successfully processed within a timeout. Storm keeps in memory a tree DAG for each tuple generated by a spout: each node in the DAG reprents a componend which is supposed to process the tuple and sends an ack after doing it. If the acker receives all the acks for a tuple then that tuple is said to be fully processed, otherwise the tuple is considered failed and the spout resposible for it should be able to regenerate it. In this way no data loss is guaranteed, but is a tuple is failed because of the timeout, then it is reprocessed twice. As said for Kafka, is a duty of the application logic to be idempotent to such events.

2.5 Elasticsearch

Chapter 3

Overview

3.1 Case: Vimond Media Solution

Vimond Media Solution [3] is an OTT (Over-the-top) services provider company headquartered in Bergen, Norway. Having customers spreaded around the world, Vimond is one of the leader in the streaming media market.

3.1.1 Vimond current architecture

3.2 Proposed solution

Chapter 4

Implementation

Chapter 5

Results and Tests

Chapter 6

Conclusions

Appendix A
Framework configurations

In this section we are going to report the followed procedures for installing and setting up the frameworks used in this thesis work.

.1 Apache Oozie

The version used for this project is the 4.2.0 (stable version - June 2015). The procedure for the installation is not trivial, as you need to build a working distro from source code. Prerequisites:

- Java 1.6+
- Apache Hadoop
- Apache Maven
- ext-2.2

Java 1.8, hadoop 2.7.0 and maven 3.3.3 were used during the installation procedure. It is assumed also that hadoop is running using the default ports setting

.1.1 Installation[1]

[1] At the time of installation I came across a bug due to a repository out of service. Refer to [5] for its solution

```
#donwload the tarball from a mirror
cd ~
wget http://apache.uib.no/oozie/4.2.0/oozie-4.2.0.tar.gz
tar -xvf oozie-4.2.0.tar.gz

#build a distro according to your java version and hadoop version
cd oozie-4.2.0/bin
./mkdistro -DtargetJavaVersion=1.8 -P hadoop-2 -Dhadoop.version=2.7.0

#configure the distro
cd distro/target/oozie-4.2.0
mkdir libext
mv /path/to/ext-2.2.zip libext
bin/oozie-setup.sh sharelib create -fs hdfs://localhost:9000
bin/oozie-setup.sh prepare-war
bin/ooziedb.sh create -sqlfile oozie.sql -run
bin/oozied.sh start
```

In order to set up the integration with hadoop, the following properties need to be defined:

```
<!-- oozie-site.xml -->
<property>
    <name>oozie.service.HadoopAccessorService.hadoop.configurations</name>
        <value>*=/path/to/hadoop-2.7.0/etc/hadoop</value>
</property>
<property>
     <name>oozie.service.WorkflowAppService.system.libpath</name>
     <value>hdfs:///user/${user.name}/share/lib</value>
</property>

<!-- hadoop core-site.xml -->
<property>
       <name>hadoop.proxyuser.myhadoopusername.hosts</name>
       <value>*</value>
</property>
<property>
       <name>hadoop.proxyuser.myhadoopusername.groups</name>es
       <value>*</value>
</property>
```

Once oozie is started, is it possible to control the job executions via web interface at *localhost* : 11000

List of Figures

Bibliography

[1] Google trends *"http://www.google.com/trends/explore#q=big%20data'* (website)

[2] Ward J. S., Barker A. *"Undefined by data: A survey of Big Data Definitions'* (website)

[3] Vimond Media Solution *"http://www.vimond.com"* (website)

[4] Nathan Martz, *"How to beat the cap theorem"* (blog article) http://nathanmarz.com/blog/how-to-beat-the-cap-theorem.html

[5] BIGTOP-1875, *"https://issues.apache.org/jira/browse/BIGTOP-1875"*

[6] Martz Nathan, Warren James *"Big Data: Principles and Best Practices of Scalable Realtime Data Systems'* Manning, 2015 (book)

[7] Gilbert Seth, Lynch Nancy: *"Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services'* 2002 (paper)

[8] Heule Stefan, Nunkesser Marc, Hall Alexander: *"HyperLogLog in Practice: Algorithmic Engineering of a State of The Art Cardinality Estimation Algorithm'* Proceedings of the EDBT 2013 Conference, ACM, Genoa, Italy 2013 (paper)

[9] Boykin O., Ritchie S., O'Connel I., Lin J.: *"Summingbird: A Framework for Integrating Batch and Online MapReduce Computations'* 2013 (paper)

[10] Zaharia M. et. Al : *"Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing'* In Proceedings of the 9th USENIX conference on Networked Systems Design and Implementation 2012 (paper)

[11] Kreps Jay, *"Questioning the Lambda Architecture'* 2014 (blog article) http://radar.oreilly.com/2014/07/questioning-the-lambda-architecture.html

[12] Kreps Jay, Narkhede Neha, Rao Jun *"Kafka: a Distributed Messaging System for Log Processing'* 2011 (paper)

[13] Apache oozie *"http://oozie.apache.org/"* (website)

[14] Zaharia M. et Al. *"Spark: Cluster Computing with Working Sets"* 2013 (paper)

[15] Zaharia M. et Al. *"Resilient Distributed Dataset: A Fault-Tolerant Abstraction for In-Memory Cluster Computing"* 2013 (paper)